After running

```
cd "$HOME/csce465-agentsec"
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install cryptography==49.0.0 pytest==9.1.1
```

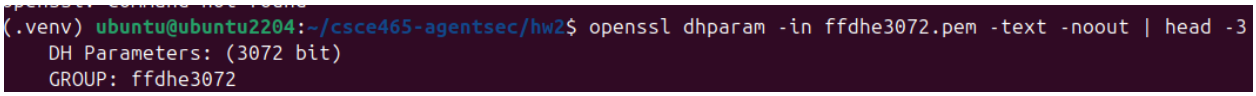
It told me that I did not have the “python3-venv” package. Thus, I installed it and ran again.

Running

```
cd "$HOME/csce465-agentsec/hw2"
openssl genpkey -genparam -algorithm DH -pkeyopt group:ffdhe3072 -out
ffdhe3072.pem
```

Also returned similar results, with needing to create the hw2 folder.

Confirming a correct installation,



Was returned.

Lab Preperation:

(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3

(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)

(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:

Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laugher, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:

Part 1

I installed VIM because I wanted to.

Then, looking at this website explaining how to do AES-CTR without mac: https://cryptobook.nakov.com/symmetric-key-ciphers/aes-encrypt-decrypt-examples

Then, I encrypted a ‘data.json’ file, and decrypted it to make sure it worked.

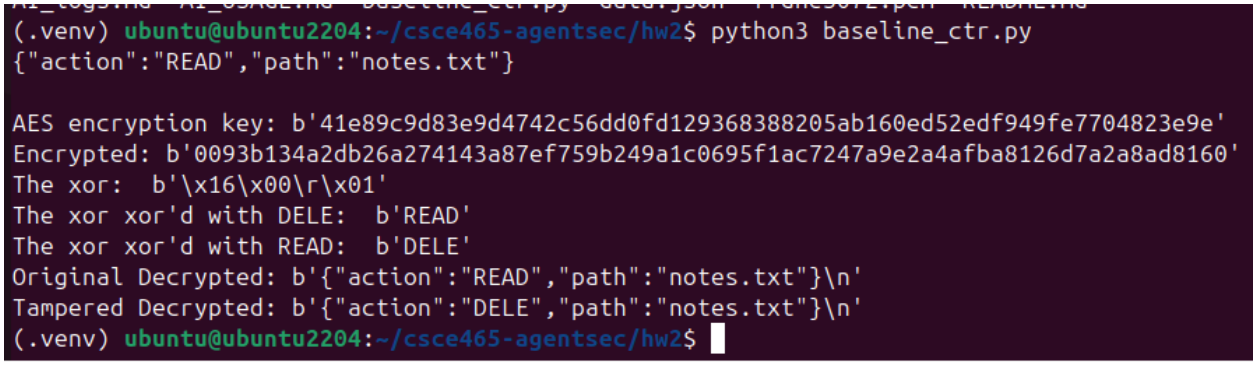
Then, I was looking at how I should XOR a READ and DELE byte string, so that I could xor the ciphertext at a specific point to get back a readable plaintext. I first created a XOR_string function to xor strings together, but it didn’t work with the ciphertext. Thus, with the help with perplexity to write a XOR_byte function, I was able to successfully xor READ and DELE, and recover either one by XOR’ing again.

Then, using some help from perplexity to iron out a couple of bugs, I was able to launch an attack on the AES-CTR encryption, successfully turning the “READ” into a “DELE”. Running the program gives this:

(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 baseline_ctr.py
{"action":"READ","path":"notes.txt"}

AES encryption key:
b'41e89c9d83e9d4742c56dd0fd129368388205ab160ed52edf949fe7704823e9e'
Encrypted:
b'0093b134a2db26a274143a87ef759b249a1c0695f1ac7247a9e2a4afba8126d7a2a8ad8160'
The xor: b'\x16\x00\r\x01'
The xor xor'd with DELE: b'READ'
The xor xor'd with READ: b'DELE'
Original Decrypted: b'{"action":"READ","path":"notes.txt"}\n'
Tampered Decrypted: b'{"action":"DELE","path":"notes.txt"}\n'

The original was decrypted successfully, and the tampered also successfully recovered a “DELE” instead of a “READ”.



Task 2

For this task, I heavily used perplexity to generate code and to follow along with task 2. However I will attempt to best follow along with the steps outlined in Task 2 and fit them along into the generated code.

1. Using the requirements from number 1, the following protocol constants have been derived:

```
# ---------------------------------------------------------------------
# Protocol constants required by the assignment
# ---------------------------------------------------------------------

PROTOCOL_LABEL = b"CSCE465-HS-v2"
GROUP_IDENTIFIER = b"ffdhe3072"

ROLE_GATEWAY = b"gateway"
ROLE_NODE = b"node"

KDF_LABEL = b"CSCE465-KDF-v1"

GATEWAY_ID = "gateway-01"
NODE_ID = "node-01"

DH_WIDTH = 384          # ffdhe3072: 3072 bits / 8 = 384 bytes
NONCE_WIDTH = 16        # Assignment requirement
```

In order to generate RSA keys,

```
from cryptography.hazmat.primitives.asymmetric import rsa
```

Was used.

RSA keys were created in code using

```
def generate_rsa_signing_key():
    return rsa.generate_private_key(
        public_exponent=65537,
        key_size=3072,
        

gateway_rsa_private = generate_rsa_signing_key()
node_rsa_private = generate_rsa_signing_key()

gateway_rsa_public = gateway_rsa_private.public_key()
node_rsa_public = node_rsa_private.public_key()
```

Which was then saved to the folder under .pem files under the names “gateway” and “node”.

2. Then, to keep each session fresh and to deter replay attacks, fresh private values are generated pulling from the standardized ffdhe3072 group (as well as nonces we generated).

The nonces generated are used to make sure a session is “fresh” / mainly used to make sure that a session isn’t being reused at another time. This prevents replay attacks, since a replay from an old session has the wrong gateway nonce / public value to the current pending handshake.

DH keypairs for each session are generated here,

```
def generate_ephemeral_dh_keypair(parameters):
    private_key = parameters.generate_private_key()

    public_y = private_key.public_key().public_numbers().y
    public_bytes = int_to_384_bytes(public_y)

    return private_key, public_bytes
```

And nonces are created here.

```
def generate_nonce() -> bytes:
    return os.urandom(NONCE_WIDTH)
```

3. Then, loading the group file ffdhe3072.pem, we construct a transcript containing:

    1. b"CSCE465-HS-v2"
    2. b"ffdhe3072"
    3. UTF-8 gateway identity
    4. UTF-8 node identity
    5. Gateway DH public value, exactly 384 bytes
    6. Node DH public value, exactly 384 bytes
    7. Gateway nonce, exactly 16 bytes
    8. Node nonce, exactly 16 bytes

With each field encoded in 4-byte big-endian integers.

The first is the protocol label CSCE465-HS-v2, the second the group identifier ffdhe3072, and 3&4 representing both party identities respectively.

5&6 are both ephemeral Diffie–Hellman public values, and 7&8 are both random nonces relating to the gateway and node nonces.

They are encoded through this method:

```
def encode_field(field: bytes) -> bytes:
    if not isinstance(field, bytes):
        raise TypeError("Transcript field must be bytes")

    if len(field) > 0xFFFFFFFF:
        raise ValueError("Field too large")

    return len(field).to_bytes(4, "big") + field
```

Then, these fields are validated in

```
def validate_transcript(transcript: bytes) -> dict:
    fields = parse_transcript(transcript)

    (
        protocol_label,
        group_identifier,
        gateway_id_bytes,
        node_id_bytes,
        gateway_public,
```

```
    node_public,
    gateway_nonce,
    node_nonce,
) = fields

if protocol_label != PROTOCOL_LABEL:
    raise ValueError("Wrong protocol label")

if group_identifier != GROUP_IDENTIFIER:
    raise ValueError("Wrong DH group identifier")

if len(gateway_public) != DH_WIDTH:
    raise ValueError("Gateway public value width is invalid")

if len(node_public) != DH_WIDTH:
    raise ValueError("Node public value width is invalid")

if len(gateway_nonce) != NONCE_WIDTH:
    raise ValueError("Gateway nonce width is invalid")

if len(node_nonce) != NONCE_WIDTH:
    raise ValueError("Node nonce width is invalid")

try:
    gateway_id = gateway_id_bytes.decode("utf-8")
    node_id = node_id_bytes.decode("utf-8")
except UnicodeDecodeError as exc:
    raise ValueError("Identity cannot be decoded as UTF-8") from exc

return {
    "gateway_id": gateway_id,
    "node_id": node_id,
    "gateway_public": gateway_public,
    "node_public": node_public,
    "gateway_nonce": gateway_nonce,
    "node_nonce": node_nonce,
}
```

So that no information leaks if any length is incorrectly declared.

4. The transcript is signed in this function:

```
def sign_transcript(private_key, role: bytes, transcript: bytes) -> bytes:
    th = transcript_hash(transcript)
    signed_message = role + th

    return private_key.sign(
        signed_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.DIGEST_LENGTH,
        ),
        hashes.SHA256(),
    )
```

And are implemented here:

```
th = transcript_hash(transcript)
signed_message = role + th
```

Then, it verifies the peer signature here:

```
def verify_transcript_signature(
    trusted_peer_public_key,
    expected_role: bytes,
    transcript: bytes,
    signature: bytes,
):
    th = transcript_hash(transcript)
    signed_message = expected_role + th

    try:
        trusted_peer_public_key.verify(
            signature,
            signed_message,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.DIGEST_LENGTH,
            ),
            hashes.SHA256(),
        )
    except InvalidSignature as exc:
        raise HandshakeError(
            "RSA-PSS signature verification failed"
        ) from exc
```

Specifically here:

expected_role || SHA-256(transcript)

A rejection test is compiled here:

```
expect_rejection(
    "reflected gateway signature rejected as node signature",
    lambda: verify_transcript_signature(
        gateway_rsa_public,
        ROLE_NODE,
        transcript,
        gateway_signature,
    ),
)
```

5. Then, the shared secret is computed through the two functions

```
def peer_public_key_from_bytes(
    peer_public_bytes: bytes,
    p: int,
    g: int,
):
    y = validate_peer_dh_public(peer_public_bytes, p)

    parameter_numbers = dh.DHParameterNumbers(p, g)
    public_numbers = dh.DHPublicNumbers(y, parameter_numbers)

    return public_numbers.public_key()


def compute_dh_shared_secret(
    local_private_key,
    peer_public_bytes: bytes,
    p: int,
    g: int,
) -> bytes:
    peer_public_key = peer_public_key_from_bytes(
        peer_public_bytes,
        p,
        g,
    )

    raw_secret = local_private_key.exchange(peer_public_key)
```

```
    z_int = int.from_bytes(raw_secret, "big")
    z = int_to_384_bytes(z_int)

    return z
```

And thus compute the same Z through using

```
z_gateway = compute_dh_shared_secret(
    gateway_dh_private,
    node_dh_public,
    p,
    g,
)

z_node = compute_dh_shared_secret(
    node_dh_private,
    gateway_dh_public,
    p,
    g,
)

assert z_gateway == z_node
assert len(z_gateway) == 384
```

Report:

Diffie-Hellman was established with private and public values. Each side, the gateway and the node, create the same “Z” value with the opposite side’s public values, and their own private DH value. An outside observer/attacker can only see the public values, but at least one private value is required to calculate Z.

Digital signatures and entity authentication, using the RSA-PSS signatures, prove that the party has the RSA private key. Because only the authorized party could create public keys that hash into a certain identity, (computed from the private key), only that authorized figure could be who they say they are. Thus, it authenticates that gateway or node.

Nonces are tied to replay resistance in that it helps prevent it by only making a handshake available once, and to that unique session. It binds a signed transcript to a single handshake attempt, and any attempts at replay will have a gateway realize that the old gateway

nonce and public value do not match what it needs currently. A change in nonce also changes TH, and thus invalidates the signature, and closes the attack surface.

Still, unauthenticated DH is still vulnerable to a man in a middle attack, as an attacker can use fake public DH values and establish separate secrets with another gateway and node. Here, however, because both DH public values and nonces are included in the signed transcript, substituting a DH value will change the TH (transcript hash) and so will change the signature, and as such easily catchable.

Reflection protection exists due to the idea that the gateway will always sign itself (b”gateway” || TH) and the node signs itself as well (b"node" || TH). The verifier will expect the peer’s identity, public key, and role. Therefore, the gateway signature will not be accepted, and requires both parties instead.

The security of long-term RSA keys (forward secrecy) is driven from the fresh ephemeral DH private values. If these are discarded after its use, then the compromise of a long time RSA private key does not reveal any previous conversations.

The keys are generated separately, and labeled as such. It generates distinct keys for each gateway-to-node encryption, gateway-to-node MAC, node-to-gateway encryption, node-to-gateway MAC, and session identification, helping to prevent accidental key reuse.

If signatures were removed, then peers could still calculate and use a DH secret, but whoever they calculate it with is unknown. Thus, a man in a middle attack could happen where someone claims to be the person they want to talk to, without actually being one. Nonces do not stop this because an attacker can relay or choose nonce values and it is not authenticated.

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 handshake.py
Handshake accepted.
Gateway identity: gateway-01
Node identity: node-01
Transcript hash: 8bd3ec56d016edf3d875536ecbc69f690d38dfa1404af1ef48a14d77a36cd700
Session ID: 2c88a47b19675319

Running rejection tests...
PASS: invalid node signature rejected
PASS: malformed length-prefixed transcript rejected before hashing
PASS: reflected gateway signature rejected as node signature
PASS: changed nonce rejected by signature verification
PASS: changed node DH public value rejected by signature verification
PASS: unexpected peer identity rejected

All required handshake checks completed.
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 handshake.py
Handshake accepted.
Gateway identity: gateway-01
Node identity: node-01
Transcript hash: 0d696d4fedb7fd38ed546ac31ab55d2138c6b55a48bceb3ee4aa82131d581374
Session ID: 78c7ec2541957fe8

Running rejection tests...
PASS: invalid node signature rejected
PASS: malformed length-prefixed transcript rejected before hashing
PASS: reflected gateway signature rejected as node signature
PASS: changed nonce rejected by signature verification
PASS: changed node DH public value rejected by signature verification
PASS: unexpected peer identity rejected

All required handshake checks completed.
```

Task 3

In this task, we are going to encrypt and then use the ciphertext to create a MAC (Message Authentication Code). To do this, I am employing the use of Perplexity to help me write secure_record.py. I have also used Perplexity to help me understand and lay out what I need to do in order to finish this task. Specifically asking questions regarding seal() and open_record().

In the program, seal() represents encryption much like the sealing of an envelope. open_record() is similar but in the opposite direction. Decryption it's just decryption honestly but it also verifies that it is valid and expected (and only decrypts if it passes those checks).

open_record():

- It first parses and validates the received information
- Then verify that the HMAC is correct
- Then finally it decrypts and returns the plaintext.

An important distinction is that the verifying HMAC comes before decryption since it would produce attacker-controlled plaintext (of which can then be used maliciously).

For each requirement we have:

Requirements:

- separate keys in each direction;

  This is satisfied through the differing K_g2n_enc/K_g2n_mac and K_n2g_enc/K_n2g_mac for gateway to node and node to gateway respectively.

- sequence numbers begin at zero in each direction;

  Sequence numbers, which are represented as SenderState and ReceiverState, start out as 0.

```
@dataclass
class SenderState:
    """
    State for one sending direction.

    A new sender state begins at sequence number zero.
    Use one SenderState object per traffic direction.
    """
    next_sequence: int = 0
```

```
@dataclass
class ReceiverState:
    """
    State for one receiving direction.

    A new receiver state expects sequence number zero.
    Use one ReceiverState object per traffic direction.
    """
    expected_sequence: int = 0
```

- the sender never reuses an IV under one key;

By using CTR mode, and by using “_build_iv(session_id, sequence)” to create an IV. Since the sequence is new each time, it never uses the same IV.

- the receiver expects the exact next sequence number;

At

```
```

```
    # Reject replays, skipped sequence numbers, and out-of-order
records.
    if parsed["sequence"] != state.expected_sequence:
        raise RecordError("Record sequence is not the expected next
sequence")
```

It raises a RecordError and rejects it whenever it becomes out of sequence.

• verify HMAC before decrypting;

```
    # Authenticate before decryption: Encrypt-then-MAC.
    _verify_tag(
        k_mac,
        header,
        iv,
        ciphertext,
        tag,
    )

    # The tag has verified, so ciphertext is authenticated and may
be opened.
    plaintext = _aes_ctr_decrypt(k_enc, iv, ciphertext)
```

The _verify_tag(...) portion (Which is where HMAC is verified) is called before decrypt is called.

• reject modified headers, modified ciphertext, replays, and messages in the wrong direction;

- For modified headers, the HMAC would not match and would be automatically rejected.
- For modified ciphertext, HMAC and the ciphertext would fail to verify before decryption is called.
- For replays, only expected_sequence is allowed, and it also advances after success. So any replay does not have the same sequence and as a result is rejected.
- For messages in the wrong direction, it clashes with the expected_direction here. It will refuse G->N as N->G traffic.

```
    # Reject records addressed to the wrong traffic direction.
    if parsed["direction"] != expected_direction:
```

```
        raise RecordError("Record is in the wrong direction")
```

• do not release plaintext after any error;

Because errors raise RecordError, decryption never finishes, and plaintext never gets computed.

• use constant-time MAC verification through the library.

It uses mac.verify(received_tag) from cryptography, which is the API and should be constant time.

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 secure_record.py
PASS: valid gateway-to-node record opened successfully
PASS: modified header
PASS: modified ciphertext
PASS: wrong direction
PASS: replayed record
All secure_record.py self-tests passed.
```

Task 4

To finish this task, I employed the use of AI.

1. valid handshake and bidirectional messages;
2. modified ciphertext;
3. modified authenticated header;
4. replayed record;
5. record reflected into the opposite direction;
6. incorrect RSA public key, invalid RSA-PSS transcript signature, or reflected handshake message.

(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 -m unittest discover -s tests -v
test_01_valid_handshake_and_bidirectional_messages
(test_security.Task4SecurityTests.test_01_valid_handshake_and_bidirectional_messages)
Requirement 1: ... Handshake accepted.
Gateway identity: gateway-01
Node identity: node-01
Transcript hash: 9d5fcb0bf9dfe322f8d8775f7cb97700ed2d1930c8efcff720cf31b6e919804a
Session ID: 2d16aeebd127a993
ok
test_02_modified_ciphertext_rejected
(test_security.Task4SecurityTests.test_02_modified_ciphertext_rejected)
Requirement 2: ... Handshake accepted.

Gateway identity: gateway-01
Node identity: node-01
Transcript hash: 27ab6fa0ee98d7ab1a9acf4fd0d5ae27b9da8b03157f96b164844b188085067f
Session ID: 0bf88edb11ed4897
ok
test_03_modified_authenticated_header_rejected
(test_security.Task4SecurityTests.test_03_modified_authenticated_header_rejected)
Requirement 3: ... Handshake accepted.
Gateway identity: gateway-01
Node identity: node-01
Transcript hash: 351b937432f24fe778cf3ef936b51b2619b8a704eb14b6f7ec7ff8a1a215567d
Session ID: 563f4a910f3d3d37
ok
test_04_replayed_record_rejected
(test_security.Task4SecurityTests.test_04_replayed_record_rejected)
Requirement 4: ... Handshake accepted.
Gateway identity: gateway-01
Node identity: node-01
Transcript hash: d12854b442424a7b5caaad771523620b002b1264fdb519fe7b311a4cbf170c4d
Session ID: f14ece3373245749
ok
test_05_reflected_record_rejected
(test_security.Task4SecurityTests.test_05_reflected_record_rejected)
Requirement 5: ... Handshake accepted.
Gateway identity: gateway-01
Node identity: node-01
Transcript hash: 5f094d2e170fcb36a59de8019cc8a2c28f6549832ae1cc5d8d4bf5fe71b1f3c2
Session ID: 4b4b3071975ad96f
ok
test_06_invalid_rsa_pss_signature_rejected
(test_security.Task4SecurityTests.test_06_invalid_rsa_pss_signature_rejected)
Requirement 6: ... Handshake accepted.
Gateway identity: gateway-01
Node identity: node-01
Transcript hash: 7da77983a0f17fbfe3708785e15a8e657f47866821317d447ab0dc61b2a822f5
Session ID: 1e8a1c91d4dd5a3b
ok

Ran 6 tests in 1.273s

OK

Report.

- states the attacker model and non-goals;

These designs for Task 1 - 3 suggest an attacker that can observe network data and intrude onto the communications between two users. Although in this instance, it would be a gateway and node instead of between two users. Since we are protecting against attacks like those in Task 1, we are assuming that the attacker is very knowledgeable, and that the attacker knows the protocol design, message format, public identities, and stuff that isn't meant to be kept secret. This could be IVs, session identifiers, sequence numbers, etc. This attacker would also have the means to observe (like observing network data), delay, drop (as in stop the attack), reorder (reordering commands to cause chaos or other outcomes), replay (playing a session at a later date), inject (man in the middle), and modify bytes that get transmitted between the simulated gateway and node. However, we do not assume that this attacker has access to the RSA's long term private signing key. We assume that the node and the gateway also protects their secrets diligently, since no ciphers are strong enough to protect against if a key gets stolen somewhere or another, if it's meant to be kept secret.

For goals that we are not focusing on in this lab are those surrounding Denial of Service resistance, availability of the CIA triad, and protection on compromised devices. This program does not detect whether multiple faulty requests are being made, and does not undergo any protections if the server is being overrun with requests or not. It also does not give any judgement if any requests are made at unreasonable times or locations, such as a log-in from another side of the world, or at midnight, since this lab only focuses on RSA and DH design aspects.

- explains why CTR requires unique IVs;

CTR mode requires unique IVs since it essentially creates a keysstream that xor's itself with the plaintext. If the same IVs are used in two different plaintexts, then the ciphertexts can be xor'd together to "cancel out" the key stream (since the keystream would also be the same), resulting in the original plaintexts xor'd with each other. This is dangerous because it suggests a link between the plaintexts, and if you know information about one plaintext, you can use it to figure out the other plaintext.

- explains why verify-before-decrypt matters;

Verifying detects integrity breaches (aka tampering). Since CTR mode is predictable in which bits will flip what in the plaintext, you could expose computers to attacker-controlled data and mess up some application parsing code. There could very well be a possibility of a SQL or prompt injection angle if used improperly. Having a verification process before this will make sure this doesn't happen.

- explains why independent encryption and MAC keys are used;

Our design uses 4 different and distinct keys:
K_g2n_enc, K_g2n_mac, K_n2g_enc and K_n2g_mac.
Called key separation and touched on in Task 2, since these keys are used for different purposes and as such should be unique. Keys that aren't connected to each other helps reduce the weakness in the circumstance that error in one key doesn't affect the other. Different keys in this scenario also help make sure communication is one-sided, in that communication can only go in one direction.

- compares the construction briefly with an AEAD mode such as AES-GCM;

AES-GCM stands for Advanced Encryption standard in Galois/Counter mode that tries to make up for the lack of integrity protection that AES has. Once AES-GCM encrypts plaintext, it generates an authentication tag that covers the ciphertext and some additional data. However it does not remove the need for nonces, direction, and more.

- explains why a valid protected record may still contain a dangerous authorized tool call.

Although an instruction may be valid, like {"action":"DELETE","path":"notes.txt"}, it might be dangerous, or come from an individual without the correct permissions to call it. Therefore, cybersecurity must not just be the CIA triad, but also make sure to apply security in authorization as well.